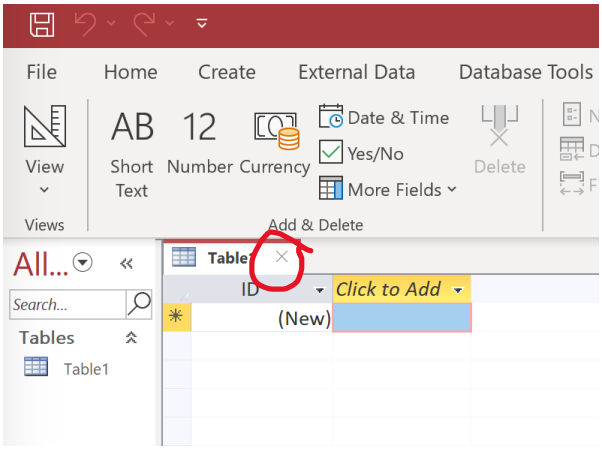
Title:	Lab 5: Basic SQL Commands
Special Notes / Additional Resources:	

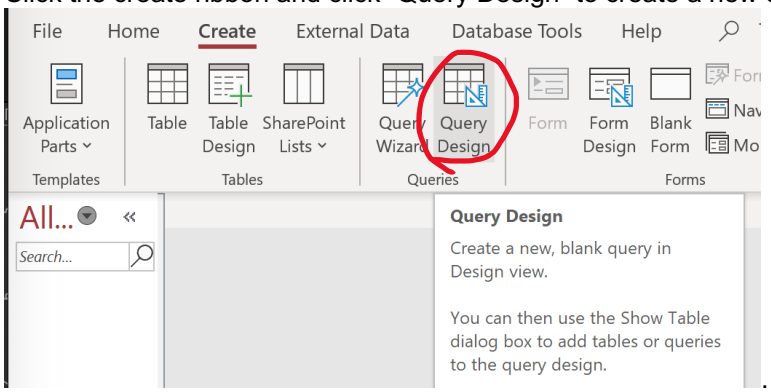
Part A: Basic Table Create and SQL Commands [5 marks]

Instructions

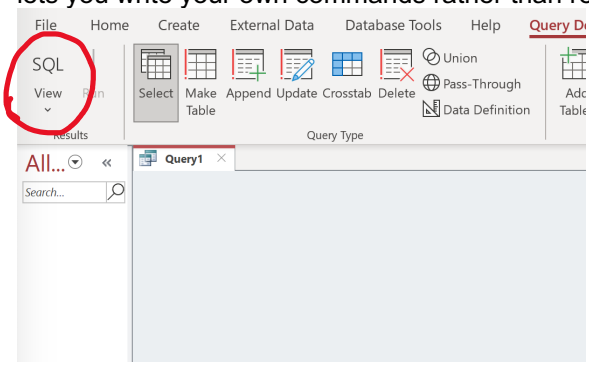
1. Create a new blank database named <YourName>-Lab-5.accdb. When Access opens, dismiss the new table wizard to continue.



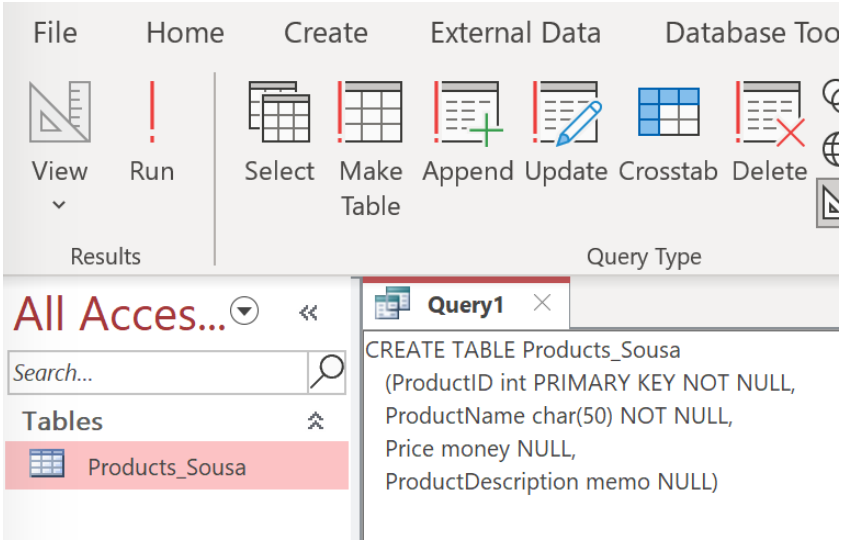
a. Click the create ribbon and click “Query Design” to create a new query.



b. Change the query design mode to SQL by clicking the “SQL” button in the ribbon. This lets you write your own commands rather than relying on Access’s Query Builder.

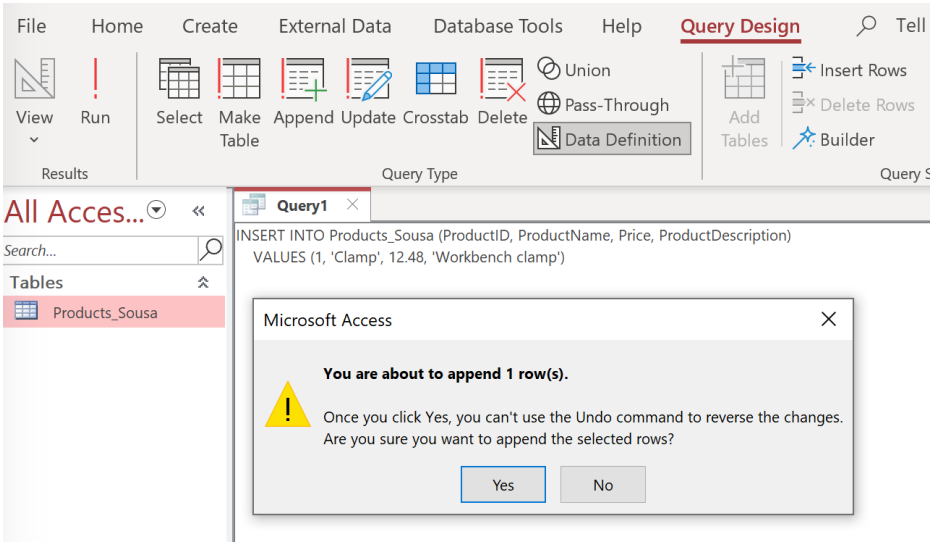


2. [2 marks]We'll now issue a command to create a table. Change the name of the table to include your last name. Click the red "Run" button in the ribbon to execute it. When successful, you will see a new table appear on the left.



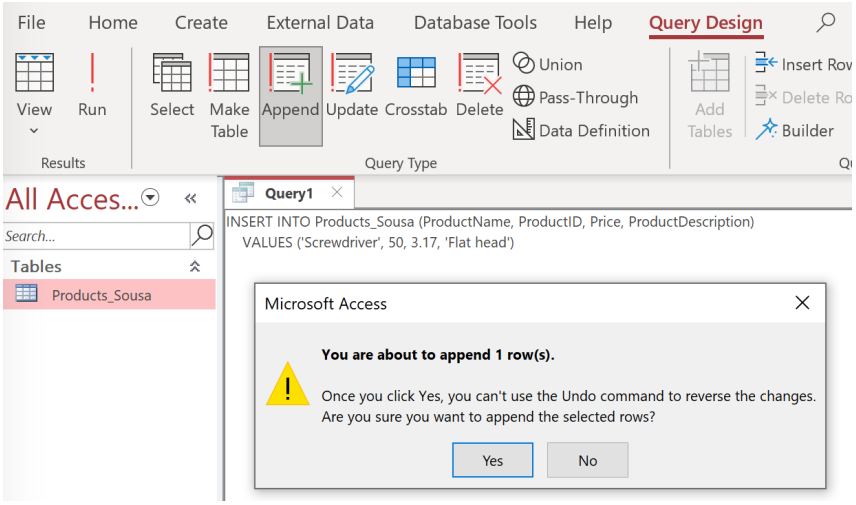
3. Insert Data Into the Table using the SQL Insert Command. We'll use a few variations of this syntax in this section

First Insert Statement (basic syntax)
Issue the statement shown (change the table name to match yours) and agree/confirm the row insert/append. Notice that this statement lists the columns and then provides the required values in the same order.



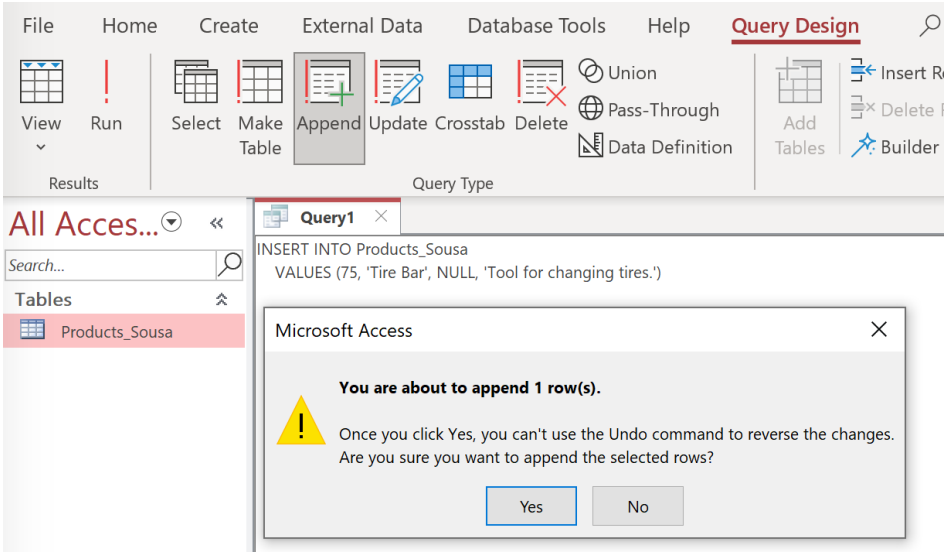
Second Insert Statement, Changing the Order of Columns

Issue the statement shown (change the table name to match yours) and agree/confirm the row insert/append. Notice that this statement lists the columns in a different order from that of the table.

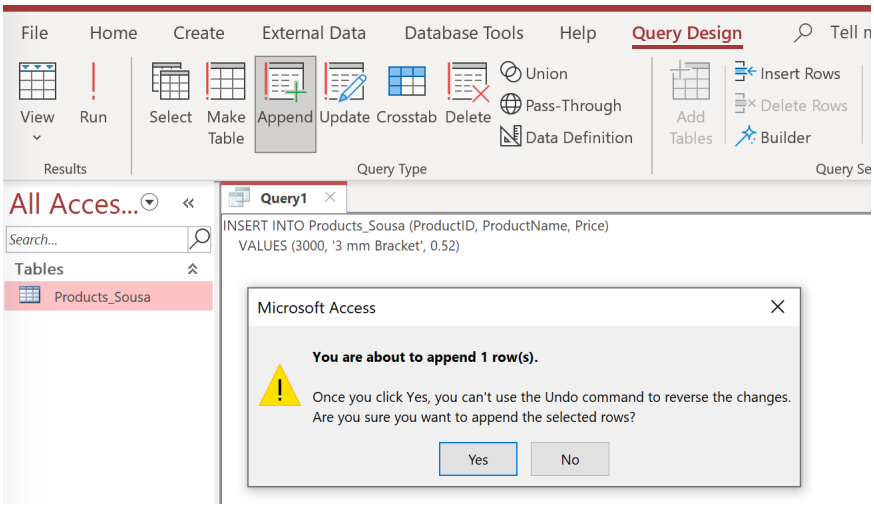


Inserting Values in the Same Order as the Columns in the Table

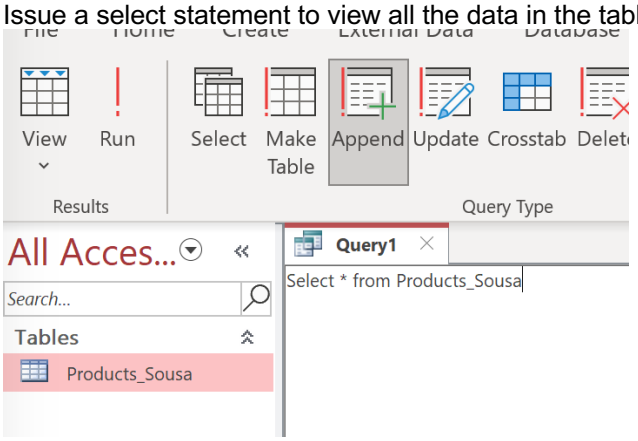
Issue the following statement (change the table to match your table name). Notice this statement only provides the list of values. The order therefore must match the order of the columns in the table.



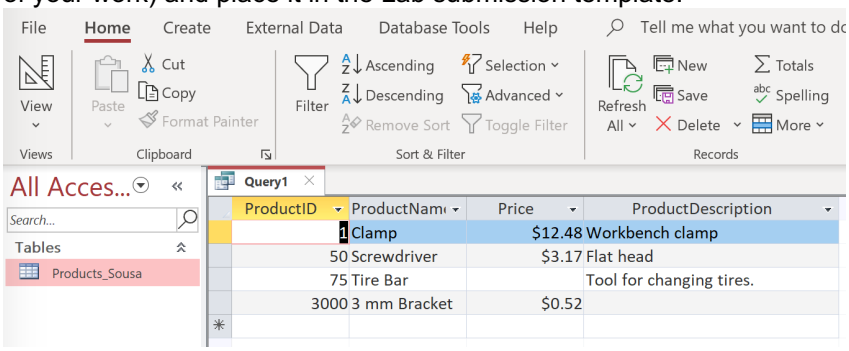
Omitting a Column During Insert
Issue the following statement (change the table to match your table name). Notice this statement omits the optional description column. Omitting a column requires that the statement include the list of explicit columns we are inserting into.



4. Issue a select statement to view all the data in the table as shown.



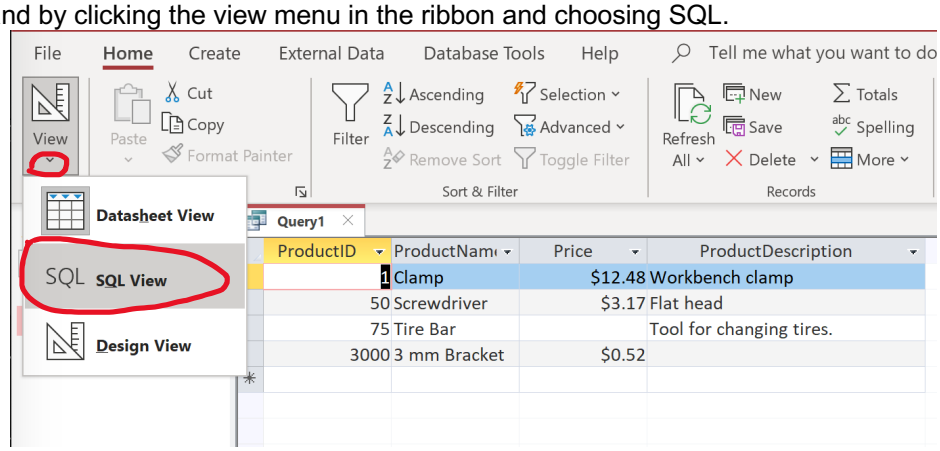
5. **[3 marks]** Take a screenshot of the data (be sure to include your visible table name as evidence of your work) and place it in the Lab submission template.



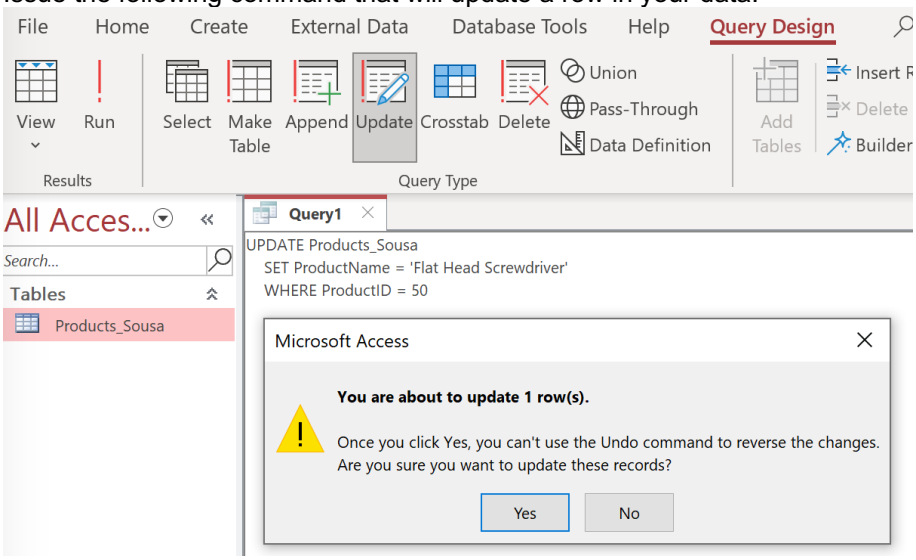
Part B: Updating and Select Query Basics [4 marks]

Instructions

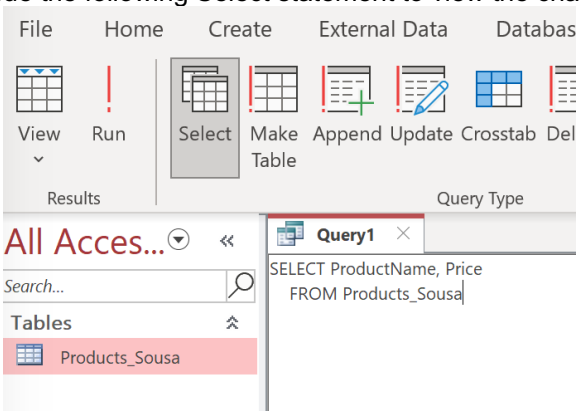
1. Continuing with the query from the last question, switch back to the SQL View to issue a new command by clicking the view menu in the ribbon and choosing SQL.



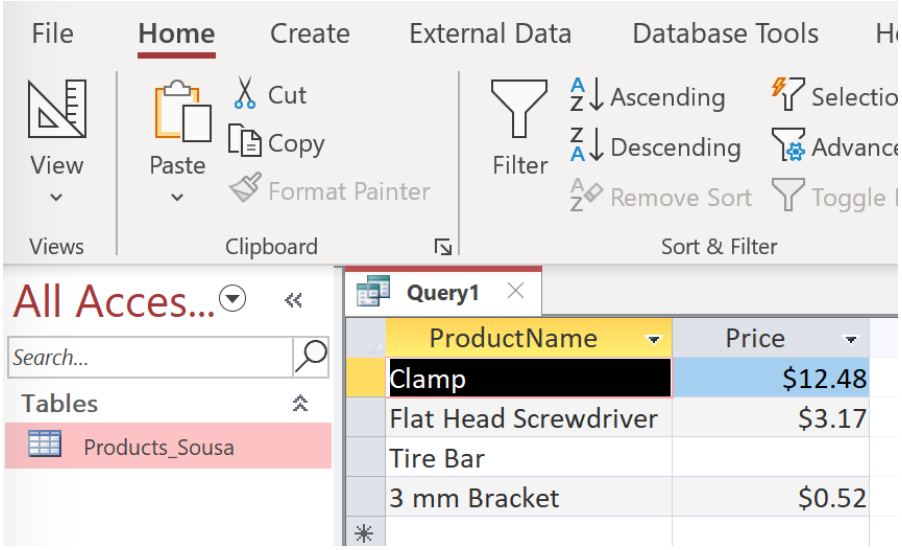
2. Issue the following command that will update a row in your data.



3. Next Issue the following Select statement to view the changed data.



4. [**2 marks]** You should see a subset of the columns similar to the screenshot below. Take a screenshot of the data (be sure to include your visible table name as evidence of your work) and place it in the Lab submission template.



5. Issue the following Select Statements taking a screenshot of each result and placing them in your submission. Notice the Select syntax is becoming more powerful with the introduction of filtering using the WHERE clause and the introduction of calculations.

a) **[1 mark]** Query with Where Clause

```
SELECT ProductID, ProductName, Price, ProductDescription
   FROM Products_Sousa
   WHERE ProductID < 60
```

b) **[1 mark]** Query with an expression/calculation in the select. Notice the “as” suffix which is used to provide a friendly column name when selecting expressions/calculations.

```
SELECT ProductName, Price, Price * 0.05 as GSTAmount
   FROM Products_Sousa
```

Part C: Aggregate Queries [5 marks]

Instructions

1. Switch back to the SQL View to issue a new command by clicking the view menu in the ribbon and choosing SQL. Issue the following commands which summarize the data in our table using SQL Aggregate functions. Be sure to take a screenshot of each step and place it in your submission template.

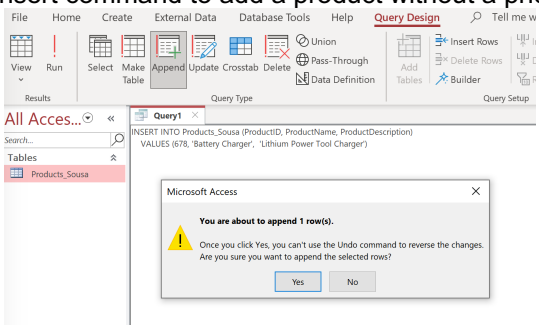
    a. [**1 mark]** Summarizing the value of products in the table using average.

    ```
    SELECT Avg(Price) as AveragePrice
       FROM Products_Sousa
    ```

    b. **[1 mark]** Counting in the table with criteria.

    ```
    SELECT Count(*) as ProductCountOverThreeDollars
       FROM Products_Sousa
       WHERE Price > 3
    ```

2. We'll now see how null values are treated by SQL queries and aggregate functions.
    a. Issue the following insert command to add a product without a price (a null price)



    b. **[1 mark]** Repeat the same query we did earlier to get the average. Notice that there is no change to the average. Aggregate functions ignore null values (they do not treat them as zero).

    ```
    SELECT Avg(Price) as AveragePrice
       FROM Products_Sousa
    ```

    c. **[1 mark]** Now we will experiment with null data specifically by issuing a count query with a filter.

    ```
    SELECT Count(*) as ProductsWithoutPrice
       FROM Products_Sousa
       WHERE Price is NOT null
    ```

    d. **[1 mark]** Now instead of counting rows using the asterisk, we'll see what count we get by counting a specific column. **Do null values get counted?**

    ```
    SELECT Count(Price) as ProductsWithoutPriceUsingCount
       FROM Products_Sousa
    ```